SMARTPICK AI

Technical Requirements Document

TRD • Architecture, data contract, algorithms, testing and deployment

SmartPick AI — Intelligent Mobile Recommender

Prepared by Pratyush Mishra
Version 1.0 • 15 October 2026
Status: Draft for review

CONFIDENTIALITY NOTE

Prepared for academic and portfolio project planning. Verify dataset rights, technical assumptions, and implementation results before external publication.

Document Control

Field	Details
Document	Technical Requirements Document (TRD)
Project	SmartPick AI — Intelligent Mobile Recommender
Version / Status	1.0 / Draft for implementation
Prepared by	Pratyush Mishra
Date	15 October 2026
Reference	SmartPick AI PRD v1.0

1. Technical Overview

This document specifies a maintainable implementation for SmartPick AI: a Python-based, content-driven smartphone recommender exposed through a Streamlit interface. The first implementation should favour transparent, testable ranking over unnecessary model complexity. Advanced ML elements are introduced only when supported by dataset quality and evaluation evidence.

2. Architecture and Module Boundaries

Module	Responsibility
app.py	Streamlit UI, state, form inputs, result rendering, charts and errors.
src/data_loader.py	Load CSV, validate required columns, normalize types and expose data metadata.
src/preprocessing.py	Missing-value policy, numeric scaling, categorical encoding, feature matrix preparation.
src/recommender.py	Budget/brand filters, weighted ranking, similarity ranking, explanation generation.
src/evaluation.py	Offline evaluation utilities and baseline comparisons; metrics require relevance labels.
tests/test_recommender.py	Unit and integration tests for filters, weights, ranking, and edge cases.
data/phones.csv	Curated data file; do not assume data is shipped until licensing is verified.

3. Proposed Repository Layout

```
SmartPick-AI/
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
■■■ data/
■   ■■■ phones.csv
■■■ src/
■   ■■■ __init__.py
■   ■■■ data_loader.py
■   ■■■ preprocessing.py
■   ■■■ recommender.py
■   ■■■ evaluation.py
■■■ tests/
■   ■■■ test_recommender.py
■■■ notebooks/
■   ■■■ eda.ipynb
■■■ assets/
    ■■■ screenshots/
```

If the dataset licence does not permit redistribution, omit the raw data from GitHub and document how an evaluator can obtain it. Keep generated files and local environments out of version control.

4. Runtime and Dependencies

Component	Recommended choice	Reason
Language	Python 3.11	Broad library compatibility; verify on the target environment.
Data	Pandas, NumPy	Tabular cleaning and numerical operations.
ML	Scikit-learn	Preprocessing, TF-IDF, cosine similarity and supported metrics.
UI	Streamlit	Python-native interactive prototype.
Charts	Plotly	Interactive charts for exploratory and comparison views.
Testing	pytest	Automated tests for deterministic core logic.
Persistence	Joblib (optional)	Persist fitted preprocessing/model objects when appropriate.
Version control	Git/GitHub	Reproducible development and portfolio publication.

5. Data Contract

The data loader should accept a CSV file and return a validated DataFrame plus metadata. The canonical schema is flexible: phone_name and price_inr are required for the MVP; additional features are optional unless a specific recommender mode requires them.

Field	Type / unit	Validation
phone_name	String	Non-empty after trimming.
brand	String/category	Optional; normalized casing and whitespace.
price_inr	Numeric, INR	Positive value; source and verification date documented.
ram_gb	Numeric, GB	Positive where present.
storage_gb	Numeric, GB	Positive where present.
battery_mah	Numeric, mAh	Positive where present.
camera_mp	Numeric, MP	Optional; not a standalone quality measure.
display_hz	Numeric, Hz	Positive where present.
has_5g	Boolean / 0-1	Normalize accepted representations.
processor_score	Numeric, source-defined	Must document benchmark, scale, and source.
source	String / URL	Record provenance where possible.
verified_at	Date	Date the source/value was last checked.

- Reject malformed required fields and report row-level issues where possible.
- Do not silently convert unknown text into numeric zero.
- Deduplicate using a documented key such as normalized brand + model name; review variants before removal.
- Keep missing values distinct from genuine zero values.
- Preserve raw source data separately if transformations need to be audited.

6. Recommendation Algorithm

6.1 MVP weighted ranking

1. Validate budget bounds and user weights.
2. Apply hard constraints such as budget and optional brand/5G filters.
3. Select the feature set available and required for the chosen mode.
4. Normalize numeric features using a fitted transformation; for a quick prototype, min-max scaling can be used with clearly documented limits.

5. Compute feature match scores or a user preference vector.
6. Combine feature scores using non-negative user weights; normalize by the sum of active weights.
7. Sort eligible candidates descending by score and return at most Top K.
8. Generate explanations from actual input features and active weights.

6.2 Content-based similarity

Create a feature matrix from scaled numeric fields and encoded categorical/text fields. Apply cosine similarity between a user profile or selected reference phone and candidate rows. Ensure price constraints are applied independently. Avoid double-counting features when combining weighted ranking and similarity; document the blend and tune it using a validation set where possible.

6.3 Reproducibility and leakage

- Fit preprocessing on the training/reference dataset, then transform evaluation data with the same fitted object.
- Do not use target/relevance labels as input features.
- Use deterministic settings where randomness is involved.
- Store feature order, preprocessing configuration, dataset version, and evaluation setup.
- Do not call a hand-written weighted formula a trained ML model; label it as a ranking baseline.

7. Evaluation Design

A recommender needs relevance judgements to measure recommendation quality. If no genuine interaction data exists, create a small, explicitly documented human-reviewed relevance set for representative user profiles. Synthetic relevance labels can test code paths but cannot establish real-world accuracy.

Metric / check	Use	Requirement
Budget compliance	Business rule	100% of displayed results must satisfy configured budget constraints.
Precision@K	Fraction of top-K items judged relevant	Requires relevance labels.
Recall@K	Fraction of relevant items retrieved in top-K	Requires a known relevant set.
NDCG@K	Ranking quality with graded relevance	Requires relevance grades or defensible labels.
Coverage	Share of catalog eligible/recommendable	Report catalog and filter context.
Diversity	Variety among results	Define a reproducible brand/model similarity rule.
Latency	Response time	Measure on target hardware and dataset size.
Robustness	Empty/malformed input behaviour	Automated tests.

8. Streamlit UI Technical Requirements

- Use sidebar or grouped form controls for budget, brand, and preference weights.
- Validate that minimum budget does not exceed maximum budget.
- Ensure weights are non-negative and at least one active weight is greater than zero.
- Display a useful empty state when filters return no rows.
- Render result cards or tables with name, price, available attributes, score, and rationale.
- Show dataset source, verification date, and missing-field caveats.
- Use Streamlit caching only for stable data/model loading; invalidate cache when dataset version changes.
- Keep ranking logic outside app.py so it can be tested independently.

9. Error Handling

Condition	Expected behaviour
Missing dataset file	Show a clear setup message; do not crash with an unexplained traceback.
Missing required columns	Report the exact missing column names.
Invalid price/feature values	Validate, flag, and exclude or correct only under a documented policy.
No eligible phones	Show a no-results message and suggest broadening filters.
All weights are zero	Ask the user to enable at least one preference.
Optional column absent	Hide/disable dependent filter or feature; do not fabricate values.
Malformed CSV	Display a controlled error and log useful diagnostic details without exposing secrets.

10. Security, Privacy, and Licensing

- No account or personal data is required for the MVP.
- Do not collect feedback persistently until storage, retention, consent, and deletion rules are defined.
- Do not store API keys in source code; use environment variables or platform secrets if future integrations require them.
- Validate any future user-uploaded CSV by size, columns, types, and file handling rules.
- Verify dataset licence, attribution, and redistribution permissions before committing data.
- Do not scrape retailer websites in violation of their terms or access controls.
- Do not commit private files, local environment folders, or personal information.

11. Testing Strategy

Test type	Examples
Unit	Budget filter, weight validation, scaling edge cases, deterministic sorting.
Data validation	Missing required columns, duplicate rows, invalid numeric values.
Integration	Load dataset → recommend → render expected result schema.
Boundary	Budget lower than every phone, equal-to-price boundary, empty dataset.
Ranking	A higher score appears before a lower score; ties use a documented secondary sort.
UI smoke test	App starts and handles a valid profile and an empty result.
Evaluation sanity	Metric output matches a small manually calculated example.

12. Deployment and Operations

1. Create and activate a Python virtual environment.
2. Install dependencies from requirements.txt.
3. Place permitted dataset at data/phones.csv or follow documented download instructions.
4. Run pytest and resolve failures.
5. Start locally with streamlit run app.py.
6. Test on a clean environment and record the Python/package versions.
7. Deploy via a supported Streamlit hosting workflow if compatible with data and dependency requirements.
8. Keep a local demo and screenshots as fallback if cloud deployment is unavailable.

```
python -m venv .venv
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

```
pytest
streamlit run app.py
```

13. Suggested requirements.txt

```
pandas
numpy
scikit-learn
streamlit
plotly
joblib
pytest
```

For a reproducible release, test and pin known-compatible versions after the app works in the target environment. The list above is a starting point, not a verified lockfile.

14. Technical Milestones

Milestone	Technical exit criteria
Data foundation	Validated schema, source notes, data checks pass.
Baseline engine	Ranking function deterministic; core unit tests pass.
UI integration	User input produces visible ranked results and empty states.
Similarity extension	Feature matrix and similarity method documented and tested.
Evaluation	Baseline comparison and supported metrics reported honestly.
Release candidate	README, dependency list, screenshots, licence notes, and demo script ready.

15. Technical Risks and Future Work

- Source data may be stale or incomplete; add provenance and refresh procedures.
- Feature weights may encode arbitrary assumptions; expose them and validate with user feedback.
- Small catalog may produce repetitive recommendations; add diversity-aware ranking.
- Cloud memory or resource limits may affect deployment; cache stable data and keep the dataset modest.
- Future improvements may include hybrid recommenders, genuine feedback, model/data versioning, scheduled data refresh, and broader device attributes.